

CI/CD Pipeline — Complete Cheat Sheet

Continuous Integration & Continuous Delivery/Deployment · Real-World Scenario · Full Keyword Glossary

DevOps

Automation

Software Delivery

Best Practices

Real Scenario

Pipeline Overview — All 7 Stages

■ Source Code



■ Build Compile



■ Test Verify



■ Security Scan



■ Artifact Package



■ Deploy Release



■ Monitor Observe

Core Definitions — CI vs CD vs Continuous Deployment

Continuous Integration (CI) — Stages 1–4

What	Developers frequently build + test run.
Goal	Catch bugs early, keep weeks of isolated work integrated.
Trigger	git push origin feature

Continuous Delivery (CD) — Stages 5–6

What	Software is always in approval.
Why	Reduces risk. Teams have no fear.
Example	Staging: auto-deploy to test

Continuous Deployment — Stages 5–7

What	Every change passing all tests is automatically deployed.
Who	Teams with very high test coverage and automation.
Example	Merge to main → live in production

Real-World Scenario — 'ShopEasy' E-Commerce App

Scenario: Developer Sara adds a Discount Code feature | React + Node.js + PostgreSQL | AWS EKS | GitHub Actions

1

Stage 1 · Source Control

Sara creates a feature branch and opens a Pull Request

Sara branches off main into feature/discount-codes. She commits her code with a clear message and opens a PR on GitHub. A webhook fires the moment she pushes, triggering the CI/CD pipeline. Her teammate reviews the code — this is the human quality gate before merge.

```
git checkout -b feature/discount-codes
git commit -m 'feat: add discount code validation at checkout'
git push origin feature/discount-codes
# webhook triggers pipeline immediately
```

Result: Pipeline triggered

2

Stage 2 · Build

GitHub Actions runner installs dependencies and compiles

A cloud runner (Ubuntu VM) checks out code. It runs npm ci (clean install from package-lock.json), runs the TypeScript compiler to catch type errors, bundles the React app, then builds a Docker image tagged with the git SHA for full traceability.

```
npm ci # exact locked deps
npx tsc --noEmit # TypeScript check
npm run build # bundle React
docker build -t shopeasy:a3f9b2c . # containerise
```

Result: Build passed · image created

3	Stage 3 • Test Three layers of automated tests run — unit, integration, E2E
Unit tests (Jest): 12 tests for discount validation — valid codes, expired, wrong format. Fast (0.8s). Integration tests: API against a real test PostgreSQL DB in Docker. E2E tests (Playwright): headless browser visits checkout, enters 'SAVE10', asserts 10% price drop. Coverage: 94%.	<pre>npm run test:unit # 12 tests · 0.8s npm run test:integration # 5 tests · 4.2s npm run test:e2e # 3 flows · 22s # Coverage: 94% on new code</pre>
Result: All 20 tests passed	
4	Stage 4 • Security Scan Snyk scans dependencies; SonarQube analyses source code
SCA (Snyk): checks 847 npm packages for CVEs. Flags lodash 4.17.20 (prototype pollution — non-critical warning). SAST (SonarQube): scans Sara's code and finds a SQL injection risk — BLOCKER. Pipeline fails. Sara fixes it using parameterised queries, re-pushes, and the scan passes.	<pre>snyk test # check npm deps for CVEs sonar-scanner # static code analysis # FAIL: SQL injection in discountController.js:47 # Sara fixes → re-pushes → passes</pre>
Result: Security scan passed after fix	
5	Stage 5 • Artifact & Registry Docker image tagged and pushed to Amazon ECR
The verified Docker image is pushed to Amazon ECR (private registry) tagged with git SHA and semver. Old images stay in the registry enabling rollbacks — if v2.4.1 breaks production, redeploy v2.4.0 in seconds. Registry is immutable — nobody can overwrite an existing tag.	<pre>docker tag shopeasy:a3f9b2c \ 123456.ecr.aws/shopeasy:2.4.1 docker push 123456.ecr.aws/shopeasy:2.4.1 # stored · immutable · ready for deploy</pre>
Result: Image published to ECR	
6	Stage 6 • Deploy ArgoCD canary rollout — 5% → 25% → 50% → 100%
Staging: ArgoCD detects the new image tag in Git (GitOps) and syncs to staging K8s cluster. Production canary: 5% of live traffic routes to v2.4.1. After 10 minutes of healthy metrics, traffic shifts to 25% → 50% → 100% over 30 min. If errors spike, ArgoCD auto-rolls back to v2.4.0 in <2 min.	<pre># staging — automatic ArgoCD GitOps sync # production canary: kubectl argo rollouts set image shopeasy \ shopeasy=123456.ecr.aws/shopeasy:2.4.1 # 5% → 25% → 50% → 100% over 30 min</pre>
Result: v2.4.1 live · 0 errors · rollout complete	
7	Stage 7 • Monitor & Observe Prometheus + Grafana tracks the new feature in real time
A Grafana dashboard shows p99 latency and error rate for the discount endpoint. Prometheus scrapes metrics every 15s. Alert rule: if error rate >1% for 5 min, PagerDuty wakes the on-call engineer. Distributed tracing (Jaeger) reveals a 200ms slow query Sara optimises next sprint. DORA: lead time dropped to 47 min.	<pre># Prometheus alert rule: - alert: DiscountEndpointErrors expr: rate(http_errors[5m]) > 0.01 for: 5m → PagerDuty notify on-call engineer</pre>
Result: p99: 42ms · error rate: 0.0% · lead time: 47min	

Pipeline Config — GitHub Actions (Annotated)

```
name: ShopEasy CI/CD
on:
  push:
    branches: [main, 'feature/**'] # trigger on any branch push
  pull_request: # also trigger on PR open
env:
  IMAGE: 123456.ecr.aws/shopeasy # no hardcoded secrets here!
jobs:
  build-test:
    runs-on: ubuntu-latest # fresh VM for every run
    steps:
      - uses: actions/checkout@v4 # clone repo at this SHA
      - run: npm ci # reproducible install
      - run: npm run lint # catch style issues early
      - run: npm test -- --coverage # unit + integration tests
      - run: npm run build # compile TypeScript / bundle
  security:
    needs: build-test # only runs if build-test passes
    steps:
      - run: snyk test --severity-threshold=high
      - run: sonar-scanner
  publish:
    needs: security
    if: github.ref == 'refs/heads/main' # publish from main only
    steps:
      - run: docker build -t $IMAGE:${{ github.sha }} .
      - run: docker push $IMAGE:${{ github.sha }}
  deploy-staging:
    needs: publish
    steps:
      - run: argocd app set shopeasy-staging --helm-set image.tag=${{ github.sha }}
```

Complete Keyword Glossary — Every Term Explained

VERSION CONTROL & BRANCHING

Git	Distributed version control system tracking every change. Each developer has the full history. <i>Example: git log shows all commits with author + timestamp</i>
Branch	Independent line of development. Branch off main, work safely, merge back when done. <i>Example: feature/discount-codes is isolated from main until done</i>
Commit	Snapshot of changes saved to Git history with a unique SHA hash. The atomic unit of version control. <i>Example: git commit -m 'fix: null pointer in checkout'</i>
Pull Request (PR)	Proposal to merge one branch into another. Enables code review and automatic pipeline checks before merging. <i>Example: Also called Merge Request (MR) in GitLab</i>
Webhook	HTTP callback that GitHub sends to your CI server the instant an event happens (push, PR open). <i>Example: This is how pipelines get triggered automatically</i>
SHA / Commit hash	Unique 40-character fingerprint (e.g. a3f9b2c) identifying an exact commit. <i>Example: Used to tag Docker images — trace exactly what code is in production</i>
Trunk-based dev	Everyone commits to main frequently (daily) using short-lived branches. Preferred for fast CI/CD. <i>Example: Branches live less than 1 day</i>
GitFlow	Branching strategy with long-lived develop, release, hotfix branches. For scheduled release cycles. <i>Example: Less ideal for continuous deployment</i>

PIPELINE STRUCTURE

Pipeline	The full automated sequence of stages (source→build→test→deploy) that runs every time code changes. <i>Example: Defined as code in a YAML file</i>
Stage	A logical phase in the pipeline (Build, Test, Deploy). Stages run in order; a failed stage stops everything. <i>Example: CI pipeline has 4 stages; full CD pipeline has 7</i>
Job	A unit of work within a stage. Multiple jobs in the same stage can run in parallel. <i>Example: unit-tests and lint run as two parallel jobs</i>
Step	A single command or action inside a job. <i>Example: run: npm test is one step</i>
Runner / Agent	The machine (VM or container) that executes pipeline jobs. <i>Example: GitHub provides cloud runners; self-host on your own servers</i>
Trigger	The event that starts the pipeline — push, PR, scheduled cron, or manual button. <i>Example: on: push in GitHub Actions YAML</i>
Needs / depends-on	Declares that one job must complete before another starts. Creates the job execution order. <i>Example: deploy-staging needs: publish</i>
Matrix build	Run the same job across multiple configurations in parallel. <i>Example: Test on Node 18, 20, and 22 simultaneously</i>
Cache	Save downloaded dependencies between runs to speed up pipelines. <i>Example: Caching node_modules saves 2-3 minutes per run</i>
Secret / Env var	Sensitive values (API keys, passwords) stored encrypted in CI settings, injected at runtime. <i>Example: Never put secrets in YAML files or source code</i>
Idempotent	Running the same pipeline step multiple times produces the same result. <i>Example: Deploy scripts must be safe to re-run without side effects</i>
Fail fast	Design principle: cheapest/quickest checks run first so failures are caught early. <i>Example: Lint and unit tests run before expensive E2E tests</i>

BUILD & PACKAGING

npm ci	Install Node.js dependencies from package-lock.json exactly — no version drift. Faster, reproducible. <i>Example: Use in CI; use npm install locally for development</i>
Lint	Static code style analysis — catches formatting errors, unused variables, anti-patterns without running code. <i>Example: Tools: ESLint, Pylint, RuboCop</i>

Static analysis	Analyse source code without executing it to find bugs, code smells, or security issues. <i>Example: Tools: SonarQube, Pylance, Checkstyle</i>
Docker image	Lightweight, standalone package containing app + runtime + dependencies. Runs identically everywhere. <i>Example: Same image on laptop, CI runner, and production server</i>
Dockerfile	Recipe text file defining how to build a Docker image step by step. <i>Example: FROM node:20 · COPY . . · RUN npm ci · CMD node app.js</i>
Semver	Semantic Versioning: MAJOR.MINOR.PATCH. v2.4.1 = breaking-change 2, new-feature 4, bug-fix 1. <i>Example: Standard way to version software releases</i>
Immutable artifact	Once built and tagged, an artifact never changes. v2.4.1 always means the same image. <i>Example: Prevents 'works on my machine' surprises in production</i>
Reproducible build	Same source + same inputs = identical output every time, on any machine. <i>Example: Achieved by pinning exact dependency versions</i>

TESTING

Unit test	Tests a single function or class in isolation. Dependencies replaced with mocks. Very fast (milliseconds). <i>Example: Tools: Jest, JUnit, Pytest — you should have thousands of these</i>
Integration test	Tests multiple components working together — e.g. API hitting a real test database. <i>Example: Slower than unit tests but catches wiring bugs between components</i>
E2E test	End-to-End: headless browser simulates a real user clicking through the full application. <i>Example: Tools: Cypress, Playwright, Selenium — slowest, highest confidence</i>
Test coverage	The % of your code executed during tests. 80% is a common minimum target. <i>Example: High coverage doesn't guarantee quality — assertions must also be meaningful</i>
Mock / Stub	Fake replacement for a real dependency (database, API) in unit tests. <i>Example: Controls what the dependency returns so you test your code, not theirs</i>
TDD	Test-Driven Development — write the test before the code. Red (fail) → Green (pass) → Refactor. <i>Example: Leads to better-designed, well-tested, maintainable code</i>
Flaky test	A test that sometimes passes, sometimes fails for no code reason (timing, random data). <i>Example: Major CI problem — erodes trust in the pipeline. Fix immediately.</i>
Test pyramid	Design principle: many unit tests, fewer integration, very few E2E. <i>Example: Optimises for speed and confidence at minimal cost</i>

SECURITY SCANNING

SAST	Static Application Security Testing — scans source code for vulnerabilities without running it. <i>Example: Catches SQL injection, XSS, insecure crypto. Tools: SonarQube, Semgrep</i>
DAST	Dynamic AST — attacks a running application like a hacker would, finding runtime vulnerabilities. <i>Example: Tools: OWASP ZAP, Burp Suite</i>
SCA	Software Composition Analysis — scans third-party libraries for known CVEs. <i>Example: Tools: Snyk, Dependabot, Black Duck</i>
CVE	Common Vulnerabilities and Exposures — public database of known security bugs with severity scores. <i>Example: CVE-2021-44228 = Log4Shell, one of the most critical ever</i>
Shift-left security	Find security issues as early as possible — in developer's IDE or CI build, not after deployment. <i>Example: Cheaper and faster to fix bugs early in development</i>
SQL injection	Attacker inserts malicious SQL into an input to manipulate the database. <i>Example: Prevented by parameterised queries. One of the most common web vulnerabilities</i>
Container scanning	Scans Docker images for known vulnerable OS packages inside the container. <i>Example: Tools: Trivy, Grype, Docker Scout</i>

DEPLOYMENT STRATEGIES

Environment	Isolated copy of the system for a specific purpose: dev → staging → production. <i>Example: Each has separate data, access controls, and configuration</i>
Blue/Green deploy	Run two identical production environments. Switch load balancer from Blue (old) to Green (new). <i>Example: Instant rollback = switch traffic back to Blue</i>

Canary deployment	Release to a small % of users first (e.g. 5%), watch metrics, then gradually increase to 100%. <i>Example: Limits blast radius of bad releases to a small fraction of users</i>
Rolling deployment	Replace old instances one by one until all run the new version. No downtime. <i>Example: Kubernetes does rolling deployments natively</i>
Rollback	Revert production to a previous known-good version when a deployment causes errors. <i>Example: Fast rollback (<2 min) is a sign of a mature CI/CD pipeline</i>
Feature flag	Deploy code to production but hide the feature behind a toggle. Enable without redeploying. <i>Example: Tools: LaunchDarkly, Flagsmith. Decouples deploy from release.</i>
GitOps	Git repo is the single source of truth for infrastructure and deployments. <i>Example: ArgoCD watches the repo and auto-syncs the cluster to match</i>
Approval gate	Manual checkpoint requiring a human to click Approve before the next stage runs. <i>Example: Used before production deploys in Continuous Delivery (not Deployment)</i>
IaC	Infrastructure as Code — define servers, networks, databases in version-controlled code files. <i>Example: Tools: Terraform, AWS CloudFormation, Pulumi</i>
Kubernetes (K8s)	Open-source container orchestration — manages deploying, scaling, and healing containers. <i>Example: Runs containers across a cluster of servers automatically</i>
Helm	Package manager for Kubernetes. Bundles K8s YAML files into reusable 'charts'. <i>Example: helm upgrade shopeasy ./chart --set image.tag=v2.4.1</i>
Immutable infrastructure	Never modify servers after deployment — destroy and replace with a new image. <i>Example: Eliminates configuration drift and 'snowflake servers'</i>

MONITORING & OBSERVABILITY

Observability	Ability to understand what's happening inside a system from its external outputs. <i>Example: Built on three pillars: Metrics, Logs, and Traces</i>
Metrics	Numerical measurements over time — CPU %, request rate, error count, latency. <i>Example: Prometheus collects metrics; Grafana visualises them on dashboards</i>
Logs	Time-stamped text records of events emitted by the app. Used for debugging specific incidents. <i>Example: Centralised with ELK Stack: Elasticsearch, Logstash, Kibana</i>
Distributed tracing	Follow a single request across multiple microservices to see exactly where time was spent. <i>Example: Tools: Jaeger, Zipkin, AWS X-Ray</i>
Latency (p99)	99th percentile response time — the slowest response 1 in 100 users experiences. <i>Example: Better than averages for detecting performance outliers</i>
Alerting	Automatically notify engineers when a metric exceeds a threshold. <i>Example: PagerDuty wakes on-call engineer if error rate > 1% for 5 minutes</i>
SLO	Service Level Objective — internal reliability target. <i>Example: SLO: 99.9% of requests respond in <200ms. Violation triggers incident.</i>
MTTR	Mean Time To Restore — average time to fix a production incident. DORA metric. <i>Example: Elite teams achieve <1 hour MTTR</i>
On-call	An engineer assigned to be reachable 24/7 to respond to production incidents. <i>Example: Teams rotate on-call duty weekly or bi-weekly</i>

DORA Metrics — Measure Pipeline Health

Deployment Frequency How often you deploy to production Elite: multiple/day · Low: monthly	Lead Time for Changes Commit to production time Elite: <1 hour · Low: >6 months	Change Failure Rate % of deploys causing incidents Elite: 0-5% · Low: 46-60%	MTTR Time to restore service after incident Elite: <1 hour · Low: >6 months
---	--	---	--

Popular CI/CD Tools

GitHub Actions CI/CD · Cloud · Free tier	GitLab CI/CD CI/CD · Built-in · Self-host	Jenkins CI/CD · Open source · Self-host
CircleCI CI/CD · Cloud-native · SaaS	ArgoCD GitOps CD · Kubernetes-native	Snyk SCA + SAST · Security scanning
SonarQube Code quality + SAST scanning	Trivy Container + IaC scanning	Prometheus Metrics collection · Open source
Grafana Dashboards + alerting	Kubernetes Container orchestration platform	Terraform IaC · Multi-cloud provisioning

Best Practice Tips

• Fail fast — put cheapest checks (lint, unit tests) first in the pipeline	• Never hardcode secrets — use encrypted CI environment variables
• Pin all dependency versions for reproducible, trustworthy builds	• Every pipeline run must be idempotent — safe to re-run without side effects
• Test in an environment that mirrors production as closely as possible	• Track DORA metrics every sprint to measure and improve pipeline health
• Use feature flags to decouple deployment from feature release	• Treat your pipeline configuration as production code — review and test it